SC2005 Lab 3 — Synchronization and Semaphores

What this lab is about

In Lab 2, you examined how xv6 selects a runnable process and switches execution between processes. In this lab, you will examine what happens when processes access the same data, implement a blocking semaphore in the kernel, and use its system-call interface to coordinate user processes.

You are encouraged to use the `CCDS VMs` for this lab. Instructions for accessing and using them are available under `Labs > Lab guidelines`. The VMs include `Visual Studio Code (VSCode)`, which provides a convenient way to browse and edit the xv6 source. Open the xv6 source directory in `VSCode` and use its file explorer and search features to locate files, find functions, and make your changes.

The shared counter and buffer are created in the kernel memory. User programs access them through system calls. Although `fork()` copies a process's user memory, parent and child can still access these same kernel objects. Their operations can interleave on one CPU or run concurrently on different CPUs, so you must identify and protect each critical section: the operations that must not overlap with conflicting accesses.

Each part has its own task, test, and checkpoint. You do not need to save logs or carry results from one part to another.

By the end of the lab, you should be able to:

- Explain why a separate check and update cannot replace a locked check and update operation.
- Trace how kernel `sleep()` and `wakeup()` coordinate a condition change with process scheduling.
- Complete a blocking semaphore and distinguish its kernel implementation from its user-facing system calls.
- Protect an entire shared-counter update against lost updates.
- Coordinate producers and a consumer using both resource counts and mutual exclusion, and explain what the supplied tests do and do not establish.

How to use Gemini Notebook in this lab

Use the course Gemini Notebook containing the Lab 3 source tree and the xv6 book. Text headed **Ask Gemini Notebook** inside a pale-blue shaded block is a prompt you can give the notebook. Other text gives instructions, explanations, required behavior, commands, or checkpoints.

Use the same evidence loop as Lab 2:

1. **Formulate expectations:** predict the result or state the invariant your implementation must preserve.
2. **Research:** ask a focused question about the supplied sources and request exact paths, functions, and line ranges.
3. **Verify in source:** open the cited locations and correct claims made by AI that do not match the `lab3` branch.
4. **Verify by execution:** build and run the test for the current part; inspect the actual result.
5. **Explain:** connect the result to the code.

Do not ask Gemini Notebook to inspect the code you write. When debugging, describe the exact compilation error, runtime symptom, or test output, and use the response as a checklist for inspecting your own work. Ask TA for guidance if needed.

Use prompt blocks when you need help; you do not need to submit AI exchanges or maintain a logbook. At each checkpoint, be ready to explain the relevant source and the result of that part's exercise. You may demonstrate the result directly; no saved output or comparison with an earlier part is required.

Before you begin — Confirm the checkout

Indicative time: 5 minutes. Use a working lab machine or VM with the xv6 toolchain prepared for the session.

Prompt	Where the command runs	Examples
`host$`	Linux lab machine or VM	`git`, `make`, `cp`
`xv6$`	xv6 shell inside QEMU	`race_test`, `forktest`

The prompt labels are explanatory; type only the command after them. Clone xv6 repository:

```
host$ git clone https://github.com/CPS-research-group/ntu-sc2005-xv6.git xv6-lab3
host$ cd xv6-lab3
host$ git status --short
host$ git fetch origin
host$ git checkout lab3
host$ git branch --show-current
host$ make clean
host$ make CPUS=3 qemu
```

Use `CPUS=3` throughout this lab so processes can execute concurrently. To leave QEMU, press Ctrl-a, release both keys, and then press x; this sequence is written as `Ctrl-a x` below. After source edits, exit QEMU and rerun `host$ make CPUS=3 qemu`.

If any test hangs or aborts, inspect the diagnostic, exit QEMU with `Ctrl-a x`, and restart the QEMU VM before retrying. An aborted test can leave allocated semaphores or unfinished child processes behind.

The executable `race_test` contains three test functions in `user/race_test.c`: `sem_basic()`, `race_test()`, and `producer_consumer()`. The supplied `main()` calls all three in order. To test a specific part independently, temporarily leave only its test call and `exit(0)` active in `main()`. Comment out the other two test calls and the `ALL TESTS PASSED` print statement; keep every function definition and assertion unchanged.

Checkpoint 0 — Orient yourself before editing

You are ready when you can:

- Confirm the `lab3` branch and a three-CPU boot.
- Locate `kernel/spinlock.c`, `kernel/proc.c`, `kernel/semaphore.c`, `kernel/ucnt.c`, `kernel/ubuf.c`, and `user/race_test.c`.
- Identify how to select one test function in `main()` for the part you are working on.

Part 1 — Understand spinlocks and sleep/wakeup

Indicative time: 25 minutes. Lab 2 focused on which process runs next. Here, you will see how scheduling and synchronization are connected: a process may need exclusive access to shared state, or it may need to wait until another process changes a condition.

Starting point: use the supplied `lab3` spinlock and process code. No semaphore implementation or `race_test` result is needed for this part.

1.1 Inspect a spinlock

Read `struct spinlock` in `kernel/spinlock.h` and the following functions in `kernel/spinlock.c`. The field indicating whether the lock is held is named `locked`.

```
void initlock(struct spinlock *lk, char *name);
void acquire(struct spinlock *lk);
void release(struct spinlock *lk);
```

When a lock is busy, `acquire()` repeatedly attempts an atomic test-and-set operation. This is called spinning: the CPU continues executing while waiting for the lock. Inspect the atomic operation and the interrupt-management calls around it. Explain why disabling interrupts on one CPU does not by itself exclude another CPU.

> **Ask Gemini Notebook**
>
> In `kernel/spinlock.c` and `kernel/spinlock.h`, identify the lock state, atomic acquisition operation, and interrupt-management calls. Help me construct an interleaving in which two CPUs both enter a critical section if acquisition uses a separate read and write. Cite the source and ask me to explain each protection; do not write replacement code.

To understand why lock acquisition must be atomic, imagine that `acquire` first checks whether `locked` is zero and then sets it to one in a separate operation. Starting with an unlocked lock, trace how two CPUs could interleave these steps. Could both CPUs believe they have acquired the lock and enter the critical section? Explain how the atomic test-and-set operation prevents this situation. Complete this as a reasoning exercise; leave the kernel implementation unchanged.

1.2 Trace a blocking wait

Read these kernel interfaces in `kernel/proc.c`:

```
void sleep(void *chan, struct spinlock *lk);
void wakeup(void *chan);
```

Kernel `sleep(chan, lk)` differs from user `sleep(ticks)`. It lets a process wait for a condition to change. The channel `chan` identifies matching sleepers; it is not a message queue or a copy of the shared data.

In xv6, checking a condition and going to sleep must be coordinated so that a wakeup is not lost between these steps. To understand how this works, trace the two lock roles in `kernel/proc.c`:

- `lk` (condition lock): protects the shared data that the caller checks and updates.
- `p->lock` (process lock): protects process `p`'s scheduling state and sleep channel.

The waiter checks its condition while holding `lk`. If it cannot proceed, it calls `sleep(chan, lk)`. Inside `sleep()`, the kernel acquires `p->lock` before releasing `lk`, sets the process to `SLEEPING`, and calls `sched()`. Before returning, `sleep()` reacquires `lk`.

The waking process holds the same condition lock while updating the shared data and calling `wakeup(chan)`. `wakeup()` acquires process locks internally; it does not acquire the condition lock for its caller.

`wakeup()` makes matching sleepers `RUNNABLE`; it neither runs them immediately nor reserves a resource. The waiter must therefore recheck its condition in a loop while holding `lk`.

> **Ask Gemini Notebook**
>
> Trace `sleep(chan, lk)` and `wakeup(chan)` in `kernel/proc.c`. Identify where the condition lock and each process lock are held, released, and reacquired. How does their ordering prevent a wakeup from being lost between checking a condition and sleeping? Which process-state transitions occur, and why must the waiter check its condition again after waking? Give source locations and guiding questions, not semaphore code.

1.3 Compare spinning with sleeping

For a hypothetical wait of 10 CPU cycles, would spinning or sleeping cost less? What about 10,000 cycles? Consider blocking, wakeup, and scheduling costs, and whether the lock holder can run on another CPU. Explain your assumptions: these numbers are not measured xv6 thresholds or universal cutoffs.

Checkpoint 1 — Explain the synchronization mechanisms

You are ready when you can:

- Compare spinning and sleeping costs, stating your assumptions.
- Identify the condition lock, wait channel, and process lock in the sleep/wakeup path.
- Explain why a woken process must recheck its condition.
- Explain why separate reads and writes cannot replace atomic lock acquisition.

Part 2 — Complete the blocking semaphore

Indicative time: 30 minutes. You will now combine a short protected update with a blocking wait. The semaphore itself is a kernel object; user processes operate on it through system calls using a semaphore identifier, `semid`.

Starting point: use the supplied semaphore skeleton in `kernel/semaphore.c` and leave `kernel/spinlock.c` unchanged.

2.1 Map the interface and the starter code

Read `kernel/semaphore.c` and the declarations in `kernel/defs.h` and `user/user.h`. The syscall handlers in `kernel/sysproc.c` read arguments and call the corresponding kernel functions. This syscall wiring is already supplied.

User-facing call	Required behavior
`sem_init(value)`	Allocate a semaphore; return its ID, or `-1` if no slot is available.
`sem_free(semid)`	Release a semaphore after its users have finished.
`sem_wait(semid)`	Wait while the value is zero, then decrement it by one.
`sem_signal(semid)`	Increment the value by one and wake processes waiting on this semaphore.
`sem_getvalue(semid)`	Read its current count under the appropriate lock.

The kernel stores semaphores in the fixed array `semaphores[MAX_SEMAPHORES]`, with 32 entries in the supplied code. `seminit()` in `kernel/semaphore.c` initializes this array during boot. It differs from user `sem_init(value)`, whose syscall handler calls `sem_alloc(value)`.

Allocation and freeing are already implemented. Successful wait, signal, and free operations return `0`; keep the supplied error checks that return `-1`. `sem_getvalue()` returns the count on success. Use non-negative initial counts in this lab.

The semaphore value must remain non-negative. A wait blocks while the value is zero and decrements it when positive. Do not combine this implementation with one that allows negative semaphore values.

Allocate semaphores before forking the processes that use them, and free them only after all users have finished. `sem_free()` marks the semaphore's array entry as unused; it does not wait for or wake processes still using it.

2.2 Complete the partial implementation

A partial semaphore implementation is provided in `kernel/semaphore.c`, including commented-out code and `TODO` comments. Uncomment the supplied lines where indicated, and complete `sem_wait()`, `sem_signal()`, and `sem_getvalue()` by following the comments.

> **Ask Gemini Notebook**
>
> Explain the `TODO` comments in `kernel/semaphore.c` and how the supplied semaphore spinlock and `sleep()`/`wakeup()` functions support them. Help me reason about the required behavior without providing completed code.

2.3 Check the basic operations

In `main()` in `user/race_test.c`, select only `sem_basic()` as described in “Before you begin.” Rebuild with `host$ make CPUS=3 qemu`, then run `xv6$ race_test`. The counter and buffer exercises do not need to be completed for this check.

The supplied `sem_getvalue()` initially returns `-1`, so the first assertion cannot pass until you implement it. The completed basic test checks the count sequence **1** → **0** → **1** across allocation, wait, and signal, then frees the semaphore. Its success message is:

```
[sem_basic] PASS: Basic semaphore operations work.
```

This is an expected string from the test source, not a captured run. Inspect your actual result. A basic pass alone does not exercise waiting on a zero count: also explain from your implementation why a waiter sleeps and rechecks the count after waking.

Checkpoint 2 — Explain the semaphore implementation

You are ready when you can:

- Distinguish user `sem_init()`, boot-time `seminit()`, and kernel `sem_alloc()`.
- Explain which lock and channel your wait and signal operations use.
- Explain why a woken waiter must recheck that the semaphore value is positive before decrementing it.
- Show the basic count sequence passing and identify what that test leaves untested.
- Explain when it is safe for these exercises to free a semaphore.

Part 3 — Protect the shared counter

Indicative time: 20 minutes. Identify and protect the complete critical section when two processes update the same counter.

Starting point: use the working semaphore from Part 2. In `user/race_test.c`, select only `race_test()` in `main()`, keeping `exit(0)` active. Comment out the other test calls and the final success print. The producer-consumer exercise is not required.

3.1 Explain the lost update

Read `ucnt_get()` and `ucnt_set()` in `kernel/ucnt.c`, and `do_work()` in `user/race_test.c`. The counter interfaces are:

```
void ucnt_set(int idx, int val);
int ucnt_get(int idx);
```

The test resets counter 0 and forks one child. Parent and child each perform `N` increments. With `N` set to 1,000, the expected total is `2*N`, or 2,000. The parent waits before checking it.

An increment in `do_work()` uses an unprotected read–modify–write sequence: `ucnt_get()` followed by `ucnt_set()`. Entering the kernel does not make the pair atomic. Explain how two processes can interleave these calls and lose an update. A final count of 2,000 in one unprotected run does not prove the code is race-free.

3.2 Protect the complete operation

Complete the semaphore-creation `TODO` in `race_test()`. Both processes' `do_work()` calls must use the same semaphore. Choose its initial count for mutual exclusion, and place wait and signal around the complete read–modify–write operation.

> **Ask Gemini Notebook**
>
> In `user/race_test.c` and `kernel/ucnt.c`, help me construct a lost-update interleaving between the parent and child. Which operations must form one critical section? Why can the child's copied `semid` still identify the same kernel semaphore? Help me reason about the initial count, protected region, and cleanup order. Do not provide the completed code.

Check allocation and semaphore-operation results. Keep the parent's `wait()` before validation and cleanup. Rebuild with `host$ make CPUS=3 qemu` and run `xv6$ race_test`. Check for a final counter of 2,000 and `[race_test] PASS: consistent counter`. Explain how your critical section prevents the lost update.

Checkpoint 3 — Defend the critical section

You are ready when you can:

- Demonstrate a lost update using two processes' reads and writes.
- Explain why protecting the two system calls separately is insufficient.
- Show the expected final count and identify the semaphore shared by parent and child.
- Explain why the parent waits before validating and freeing the semaphore.

Part 4 — Coordinate producers and a consumer

Indicative time: 25 minutes. A bounded buffer needs both exclusive access and conditions: a producer must wait for space, and a consumer must wait for data. You will coordinate these requirements with semaphores.

Starting point: retain the working kernel semaphore from Part 2. The counter exercise and its results are not prerequisites. In `main()` in `user/race_test.c`, select only `producer_consumer()`; leave the other test calls and the final `ALL TESTS PASSED` print statement commented out.

4.1 Inspect the supplied buffer and workload

Read `kernel/ubuf.c`, `user/user.h`, and `producer_consumer()` in `user/race_test.c`. The buffer interfaces implemented in `kernel/ubuf.c` are:

```
char ubuf_read(void);
void ubuf_write(char val);
```

The shared circular buffer has `UBUF_SIZE` slots; the supplied size is 16. Its read and write functions do not check whether the buffer is empty or full. Both `ubuf_read()` and `ubuf_write()` call `yield()` after accessing a slot and before updating its index. These calls allow another process to run midway through a buffer operation, exposing incorrect mutual exclusion. Protect each complete buffer operation so that another conflicting access cannot occur while the caller is descheduled.

In `kernel/ubuf.c`, `ubuf_read()` accesses and advances `head`, while `ubuf_write()` accesses and advances `tail`. The comment above `struct ubuf` incorrectly reverses these roles. Follow the function bodies.

The test creates two child producers and uses the parent as the consumer. Each producer sends 48 non-null characters in `msg`. The consumer performs `MSG_SIZE * NUM_PROD` reads, which is 96 in the supplied test. The string's terminating null byte is not sent.

4.2 Plan semaphore counts and call order

Add semaphore identifiers to the existing `struct buf_sem` in `user/race_test.c` for empty slots, filled slots, and mutual exclusion. In `producer_consumer()`, initialize these semaphores before the `fork()` loop. Derive each semaphore's initial count from the initially empty buffer of `UBUF_SIZE` slots.

For `producer()`, decide the order of `sem_wait()`, `ubuf_write()`, and `sem_signal()` calls needed to wait for an empty slot, protect the write, and signal that a filled slot is available. For `consumer()`, plan the corresponding calls around `ubuf_read()` to wait for data and signal an empty slot after reading. Explain why waiting for space or data while holding the mutual-exclusion semaphore can deadlock.

> **Ask Gemini Notebook**
>
> Using `kernel/ubuf.c` and `user/race_test.c`, help me derive the semaphore counts and order of waits and signals for empty slots, filled slots, and mutual exclusion. Explain how the `yield()` calls test mutual exclusion and why waiting for space or data while holding the mutual-exclusion semaphore can deadlock. Do not write the solution.

4.3 Implement and verify

Complete the `TODO`s in `user/race_test.c`: `struct buf_sem`, `consumer()`, `producer()`, and initialization and cleanup in `producer_consumer()`. Keep the two-producer, one-consumer test, message, loop counts, and assertions unchanged. Retain the `yield()` calls in `kernel/ubuf.c`. Check allocation and semaphore-operation results, and free the semaphores after the consumer finishes and the parent has reaped both producers.

Rebuild with `host$ make CPUS=3 qemu` and run `xv6$ race_test` with only the producer-consumer call selected. Check for `[producer_consumer] PASS: message delivered`. The consumer checks that each received byte belongs to the character set present in `msg`; it does not compare the complete received stream with an expected sequence. Use the pass result together with your argument about buffer counts, mutual exclusion, and operation ordering. Do not claim that this test alone proves the absence of duplication, loss, or reordering.

If the test fails or hangs, inspect the diagnostic and use `Ctrl-p` when the console is responsive. Exit with `Ctrl-a x` and restart the QEMU VM before retrying.

> **Ask Gemini Notebook**
>
> I am running only `producer_consumer` in `user/race_test.c`. It [fails an assertion / hangs], and the current symptom or diagnostic is ___. Based only on the supplied semaphore, buffer, and test sources, give me a diagnostic checklist for count initialization, lock lifetime, wait channels, wait/signal ordering, and cleanup. Explain what a `Ctrl-p` snapshot could show. Do not write code or infer success from one passing run.

Checkpoint 4 — Explain coordination and progress

You are ready when you can:

- Explain each semaphore's role and derive its initial value from the empty buffer.
- Trace the semaphore and buffer calls used to produce one byte and consume one byte.
- Explain why counting available slots alone does not protect the buffer's shared indices.
- Show completion with both producers reaped and the semaphores freed.
- State the limits of the supplied character-membership check.

References

- SC2005 xv6 Lab 3 branch: https://github.com/CPS-research-group/ntu-sc2005-xv6/tree/lab3.
- Russ Cox, Frans Kaashoek, and Robert Morris, *xv6: a simple, Unix-like teaching operating system*: §6.0–6.3 and §7.5–7.7 in the course edition.
- SC2005 Process Synchronization lecture.